Isolation on ...	Benefits
Deployment	Adjust deployment frequency, avoid deploying unrelated code, and scale horizontally.
Language	Different languages can be used.
Failure	Contains failures to avoid their propagation.
Space	The mobility, or location transparency, of services enables reinstantiation and horizontal scalability.
State	Strong consistency within the service.
Time	Avoids blocking and avoids propagation of failures.

B.1.4 Some challenges from isolation

As with most architectural decisions, some benefits come at a cost. A few examples are location transparency, which makes it difficult to maintain sessions between replicas, and transactions, which are difficult to implement and generally not recommended. Another drawback is data consistency. So, let’s go into a little more detail about these difficulties.

SESSIONS

Sessions can be lost due to location transparency if one service fails and another takes over. But this can happen without there being a fault. A proxy that randomly or rotationally selects from a pool of services would have the same effect. Mobility gives resilience and scalability, but state—held in a service’s memory—is lost when the proxy selects another instance of the service. To solve this problem, you first need an identifier that provides the possibility of some persistence. Then you can store the data on disk as a stateless service would. Another option is to add another layer so that the service behind the load balancer can communicate with yet another